

DesiQuery: An Evidence-Grounded Multilingual Framework for Regional Language Question Answering

Chettiyar Vani Vivekanand

*Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
vani.anjali78@gmail.com*

Manishaa G

*Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
manishaa2408@gmail.com*

Madhumitha P

*Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
madhumithapanneerselvam06@gmail.com*

Kabilesh P

*Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
kabileshpalanivelan@gmail.com*

Manikandan M

*Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
mmk.mani30@gmail.com*

Abstract—Access to reliable digital information in Indian regional languages remains challenging due to linguistic diversity, limited language-specific resources, and the tendency of large language models (LLMs) to generate unsupported information when domain-specific evidence is unavailable. The developed DesiQuery framework allows users to select or upload documents and ask questions in eight Indian languages, where BAAI/bge-m3 is used to identify semantically relevant document chunks in ChromaDB and Llama 3 generates responses through Ollama using the retrieved evidence. To improve reliability and transparency, the system restricts answer generation to retrieved evidence and provides source-document, page-level, chunk-level, and similarity information when available. Queries for which relevant information cannot be retrieved are explicitly identified rather than being answered using unsupported model knowledge. The system was evaluated using a multilingual test suite covering supported and unsupported information-seeking queries across the eight target languages, with evaluation focused on language consistency, grounding, source traceability, response completeness, and latency. The results demonstrate the feasibility of locally deployed, source-traceable RAG for regional-language document question answering while also identifying response-generation limitations for some longer multilingual outputs. The proposed framework provides a privacy-oriented and extensible approach for accessing document-based information in Indian regional languages without dependence on cloud-based LLM APIs.

Index Terms—Regional Language Processing, Question Answering, Retrieval-Augmented Generation, Large Language Models, IndicNLP, ChromaDB, Semantic Retrieval, Ollama, Natural Language Processing.

I. INTRODUCTION

An era of rapidly expanding digital information has increased the use of Question Answering (QA) systems for retrieving information from large document collections. However, reliable document-based QA remains challenging for

Indian regional languages due to differences in vocabulary, scripts, morphology, and syntax. The challenge becomes more significant when answers must be derived from specific documents such as government, administrative, or domain-specific resources.

Large Language Models (LLMs) exhibit great capabilities for language understanding and generation; however, these models may produce information which is not part of the documents provided. The challenge of extracting the relevant external sources and then feeding the LLM with these sources can be addressed by employing Retrieval-Augmented Generation (RAG). Multilingual QA requires an appropriate semantic representation across the languages for efficient retrieval and generation of grounded response. In some instances, local deployment may be necessary because of the sensitivity of the information in the documents; local execution of the retrieval and language model pipeline ensures greater control over these issues. To address these challenges, we present DesiQuery, a locally deployed multilingual document-grounded QA system based on Retrieval-Augmented Generation.

This proposed system combines multilingual semantic representations using BAAI/bge-m3, ChromaDB for vector storage and semantic retrieval, and Llama 3 with Ollama for local response generation. The system supports eight Indian languages and constrains generated responses to the retrieved document evidence. It also maintains source-document, page, chunk, and similarity metadata, allowing users to inspect the evidence associated with each generated answer. If the required information is not available in the selected documents, the system indicates that the information is unavailable rather than generating an unsupported response.

The main contributions of this work are:

- 1) **Multilingual document-grounded QA:** A multilingual question-answering system is developed for document-grounded question answering across eight Indian languages, enabling semantic retrieval and response generation in regional languages.
- 2) **Local QA with source traceability:** A streamlined local QA workflow is developed using BAAI/bge-m3, ChromaDB, and Llama 3 through Ollama. The system incorporates source document, page, chunk, and similarity metadata to provide evidence traceability for generated answers.
- 3) **Evidence-grounded evaluation:** The complete pipeline is evaluated using both supported and unsupported queries across the supported languages, considering language consistency, grounding, source traceability, response completeness, and response latency.

II. RELATED WORKS

Recent development of LLMs has brought enhancement in the capability of QA system for natural language understanding and knowledge-based response generation. But when the information is in a specialized document, the responses created by LLM can be hard to verify. RAG tackles this by extracting relevant data prior to generating a response. Khan et al. [1] suggested UQA-RAG for Urdu QA system that involved lexical and semantic retrieval, and Amal et al. [2] introduced another deterministic token-based grounding-based system named EduRAG for Urdu QA. Hindi et al. [3] discussed the use of RAG in legal domain, Xu et al. [4] introduced ChatTf with knowledge graphs for factuality and Saha et al. [5] presented QuIM-RAG by inverse question matching. Retrieval and grounding play an important role in these studies, but they concentrate primarily on single retrieval methods or retrieval areas, and not on a multi-lingual regional-language QA, local inference, and source-level traceability.

A. Multilingual and Regional-Language Question Answering

Multiple languages in a QA system pose difficulties. This is because of the differences in word usage, grammar and writing patterns. In the context Onan et al. [6] present a collection of low-resource educational applications. Reviewed the performance of cross-modal LLM QA, focusing on quality Logical and consistent answers and explainability. The work of Perevalov et al. [7] addressed the question of QA on multilingual knowledge graphs and explored the use of language models to filter candidate queries. Language-specific regional-language QA has also been investigated. datasets. Sinthusha et al. [8] were concerned with Tamil machine reading. This involves improving comprehension by making a dataset from the SQuAD corpus more accessible by translating it. Their study showed some of the challenges of implementing QA approaches. between different languages that have different linguistic features. Ali et al. [9] reviewed multilingual relation extraction and discussed challenges associated with uneven resource availability and multilingual representation.

These studies motivate the development of language-aware retrieval and evaluation approaches for regional-language QA systems, particularly for languages with limited resources and distinct linguistic structures.

B. Semantic Retrieval and Multilingual Representation

Dense retrieval uses embeddings to encode queries and documents and matches them by semantic relevance. Hambarde and Proença [10] examined sparse, dense, and hybrid information retrieval approaches and emphasized the importance of retrieval and ranking quality.

Multilingual embedding models enable representations from multiple languages to be mapped into a shared semantic space, supporting cross-lingual retrieval. DesiQuery uses BAAI/bge-m3 embeddings for multilingual semantic matching, which is stored in the local vector repository called ChromaDB.

However, retrieving a relevant document chunk alone does not guarantee that the generated response will remain faithful to the retrieved evidence. Therefore, semantic retrieval must be combined with evidence grounding and response verification to improve the reliability of multilingual QA.

C. LLM Hallucination and Response Reliability

Hallucination and unsupported generation remain significant limitations of LLM-based QA systems. Hu et al. [11] explored how to incorporate outside knowledge into VQA and highlighted the importance of grounded information. A multi-perspective consistency method to identify hallucinations in LLM was introduced by Kong et al. [12]. Behmanesh et al. [13] investigated confidence-based answer selection for knowledge-based question answering.

These studies demonstrate that improving the language model alone is not sufficient to achieve reliable QA. Retrieval quality, evidence verification, and grounding of generated answers are also important. The proposed DesiQuery system addresses these challenges by restricting response generation to retrieved document evidence and retaining source and page metadata for traceability.

D. Local and Resource-Constrained LLM Deployment

Local deployment introduces additional constraints related to model size, inference time, memory requirements, and available computational resources. Cerqueira [14] conducted a quantitative evaluation of quantized local LLMs for RAG under resource-constrained settings, considering answer quality, retrieval performance, and runtime behavior. Choi et al. [15] investigated the generation of quality test questions and answers using quantized low-rank adapters in LLMs. Liao et al. [16] explored cross-attention reprogramming for automatic speech recognition by connecting discrete speech units with pretrained language models.

These studies illustrate the importance of efficient deployment and model adaptation under resource constraints. In contrast, DesiQuery uses BAAI/bge-m3 embeddings, local ChromaDB retrieval, and Llama 3 through Ollama without

requiring fine-tuning or cloud-hosted generation. This architecture enables the document knowledge base to be updated without retraining the language model.

TABLE I
COMPARATIVE ANALYSIS OF DESIQUERY AND EXISTING WORKS

Study	Main Focus	Retrieval Knowledge Method	Language Domain	Local Generation	Source Traceability
Khan et al. [1]	Urdu Question Answering	Lexical and semantic retrieval	Urdu	Not the primary focus	Limited
Amal et al. [2]	Explainable educational QA	Multimodal RAG and token grounding	Educational	Not the primary focus	Grounding-based
Hindi et al. [3]	RAG in legal technology	Retrieval-based generation	Legal domain	Not the primary focus	Interpretability-focused
Xu et al. [4]	Factuality-aware QA	Knowledge graph enhanced retrieval	Traditional folklore	Not the primary focus	Knowledge-graph based
Saha et al. [5]	Retrieval improvement	Inverted question matching	General QA	Not the primary focus	Not the primary focus
Cerqueira et al. [14]	Local RAG deployment	Retrieval with local LLM	Resource-constrained setting	Yes	Not the primary focus
DesiQuery	Regional-language document QA	BGE-M3 ChromaDB	+ Eight Indian languages	Llama 3 + Ollama	Document, page, chunk and similarity

The existing research has paved the way for RAG, Multilingual QA, Semantic Retrieval, Hallucination Detection, and Local LLM Deployment, but they are studied in isolation or in limited domains. DesiQuery integrates multilingual BAAI/bge-m3 embeddings, retrieval with ChromaDB, Llama 3 (with Ollama) and source-level information for document QA in regional language. System evaluation is conducted over eight Indian languages for both supported and unsupported queries, based on language consistency, language grounding, source traceability, response completeness and latency.

III. PROPOSED SYSTEM

DesiQuery is a multilingual, document-grounded question-answering framework designed for local deployment. It provides answers in eight Indian languages while ensuring that responses are generated only from the information contained in the selected documents. Users can select pre-loaded documents or upload PDF, TXT, and DOCX files and submit queries in any supported language. The system architecture is divided into modular stages covering document processing, language handling, semantic retrieval, response generation, and source attribution.

The developed system consists of six major modules: (1) Document Collection and Preprocessing, (2) Language Processing and Normalization, (3) Document Chunking and BGE-M3 Embedding, (4) ChromaDB and Semantic Retrieval, (5) RAG with Ollama and a Local LLM, and (6) Source Document and Page Number Tracking. The modular design allows individual components to be replaced, inspected, and diagnosed independently without duplicating their responsibilities.

A. Document Collection and Preprocessing

The system accepts PDF, TXT, and DOCX documents that may either be pre-loaded or uploaded by the user. During ingestion, text is extracted from the selected documents and associated with the corresponding document name and page number whenever available. Documents containing insufficient

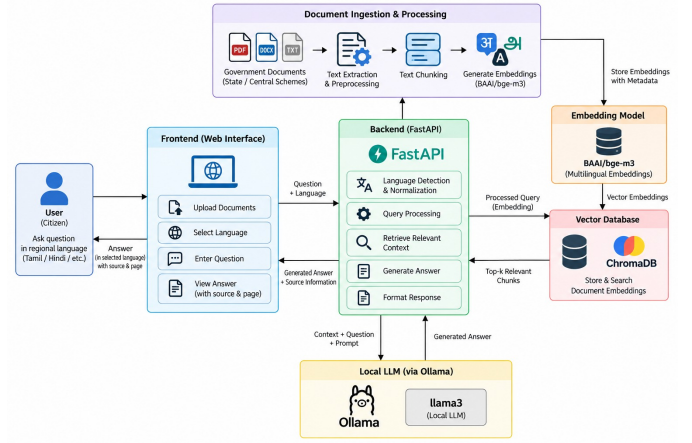


Fig. 1. System Architecture

extractable text, such as image-only PDFs, are rejected during the ingestion stage. The preserved document and page metadata are subsequently used during retrieval and source attribution.

B. Language Processing and Normalization

DesiQuery supports eight different Indian Regional languages. The user selects the language before submitting a query, and the selected language is used throughout the query processing and answer generation stages. The generated response is also expected to remain consistent with the selected language. Language consistency is subsequently validated during system evaluation.

C. Document Chunking and BGE-M3 Embeddings

The extracted document text is segmented into smaller chunks to enable fine-grained semantic retrieval. Each chunk retains its associated document identifier, chunk identifier, and page information whenever available. The BAAI/bge-m3 embedding model is used to generate multilingual semantic representations of both document chunks and user queries. The resulting embeddings are stored along with their metadata for subsequent retrieval.

D. ChromaDB and Semantic Retrieval

ChromaDB is used as the local vector database for storing document embeddings and associated metadata. When a user submits a query, its semantic representation is generated using the same embedding model and compared with the indexed document representations. The most relevant chunks are retrieved and ranked according to their semantic similarity. The retrieved evidence and associated metadata are retained for context construction and source attribution.

E. RAG and Local LLM Generation

The retrieved evidence is supplied as context to Llama 3 through Ollama for evidence-constrained generation. The generation process is constrained to the retrieved document

evidence and the selected output language. If the required information is not present in the retrieved evidence, the system indicates that sufficient information is not available rather than generating unsupported content.

Context selection is performed before generation to retain the most relevant evidence while controlling the amount of information provided to the local language model. This method is used to minimize unsupported generation by selecting the most pertinent retrieved evidence for the generation of a response.

F. Source Document and Page Number Tracking

The system maintains source information associated with the retrieved chunks, including the source document, page number, chunk identifier, and retrieval similarity score when available. The generated answer is returned together with its supporting source information, allowing users to inspect the evidence used for answer generation. The similarity score is presented as a retrieval indicator and is not interpreted as a probability or confidence score.

G. Overall Processing Pipeline

The overall processing pipeline begins when the user selects or uploads documents through the Document Upload interface and initiates text extraction. The extracted content is processed, normalized, divided into chunks, and converted into BGE-M3 embeddings. The embeddings are then indexed in ChromaDB for semantic retrieval. When a query is submitted, its embedding is generated and compared with the indexed document representations. Relevant evidence is selected and supplied to Llama 3 through Ollama for grounded answer generation. Finally, the answer is returned together with the corresponding source attribution.

The complete processing flow is represented as:

Document Upload → Text Extraction → Language Processing
→ Chunking → BGE-M3 Embedding → ChromaDB
→ Semantic Retrieval → Context Selection
→ Llama 3/Ollama → Answer + Source Attribution

This modular architecture separates document preparation, semantic retrieval, response generation, and evidence traceability, enabling pipeline-level monitoring and diagnosis of individual processing stages.

IV. METHODOLOGY

The document grounded multilingual QA system, Desi Query, is a retrieval + generation based approach that employs locally hosted LLM.

Desi Query:

- ingests content in eight languages (English, Tamil, Hindi, Telugu, Malayalam, Marathi, Bengali, Kannada),
- normalizes and chunks it,
- applies multilingual embedding and,
- performs vector storage and semantic retrieval,
- selects context from the retrieved data,

- generates answer using locally hosted LLM, validates answer to language of concern, and,
- attributes source to generated answer.

A. Document processing chunking:

Desi Query accepts PDF, TXT, and DOCX documents. Supported PDF files are retained in page granularity if page number information can be retrieved reliably from the file. Text-only documents with very limited content are ignored. Content collection D is represented as

$$D = \{d_1, d_2, \dots, d_n\}$$

Document chunks are stored for future retrieval,

$$d_i \rightarrow \{ci_1, ci_2, \dots, ci_m\}$$

Chunk ID, Page Number are preserved to support source attribution with output chunks.

B. Embedding semantic retrieval:

Both document chunks and queries are transformed to vectors using BAAI/bge-m3,

$$e_i = f_{\text{BGE-M3}}(c_i), \quad e_q = f_{\text{BGE-M3}}(q)$$

Vectors are stored in ChromaDB, and chunks similar to query are identified by cosine similarity:

$$\text{sim}(q, c_i) = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}$$

Ranking based on this measurement yields the evidence set. Sentence based filtering to remove redundant information is performed to improve quality of the content before text generation.

C. Retrieval-augmented generation:

Contextually selected evidence is combined with query and fed to Llama 3 via Ollama:

$$A = G(q, C'_q)$$

where A is answer, q is question, C'_q is filtered context, and G is the locally hosted LLM. Llama is instructed to respond in the language of interest, use only provided evidence, avoid unsupported statements and, communicate when information cannot be found in documents.

D. Validating attributing output:

The output is validated for content and language, and context source information, including document name, page number, chunk ID, and similarity score, is provided with the answer. In particular, similarity scores are indicators of retrieval relevance, rather than probabilities of answer correctness. When evidence for answering a question is not available, the answer is reported as "answer not available". In addition, generation truncation flag is observed and, in case of a truncated answer, a new prompt with a larger maximum output token count is sent to Llama.

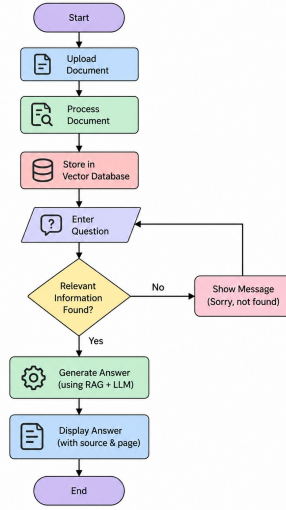


Fig. 2. Algorithm Workflow

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Setup

The DesiQuery system was evaluated to assess its performance in document-grounded multilingual question answering. The evaluation focused on language consistency, evidence grounding, source traceability, response completeness, and response latency.

A total of 24 test cases were constructed using three question categories: procedure-based questions, application/document-related questions, and unsupported questions. Each category contained one test case for each of the eight supported languages. Thus, each language was evaluated using three test cases.

The evaluation was performed through the FastAPI query interface using the complete processing pipeline, including query embedding, semantic retrieval with ChromaDB, context selection, and local Llama 3 generation through Ollama. Retrieval metadata was also verified to ensure that generated responses could be traced to the retrieved document evidence.

TABLE II
MULTILINGUAL EVALUATION TEST SET

Category	EN	TA	HI	TE	KN	ML	BN	MR	Total
Procedure	1	1	1	1	1	1	1	1	8
Application/Documentation	1	1	1	1	1	1	1	1	8
Unsupported	1	1	1	1	1	1	1	1	8
Total	3	3	3	3	3	3	3	3	24

B. Evaluation Criteria

Five criteria were considered during evaluation. Language consistency checks whether the response follows the selected output language. Grounding assesses whether statements, numerical details, and procedural instructions in the response can be traced to the selected document content. Source traceability verifies the consistency of document, chunk, and page metadata when available. Response completeness checks

whether the answer terminates naturally within the configured generation limits. Response latency indicates the time taken to process a query and generate the corresponding answer.

The evaluation was implemented using automated validation together with diagnostic information produced by the backend. Grounding validation specifically measures support from the retrieved evidence and is not treated as a general measure of factual accuracy.

C. Results and Analysis

The evaluation produced 19 PASS, 5 WARNING, and 0 FAIL results across the 24 test cases. Language validation, grounding validation, and source traceability achieved successful validation for all evaluated cases. The warning cases were associated primarily with response completion under the current local generation configuration.

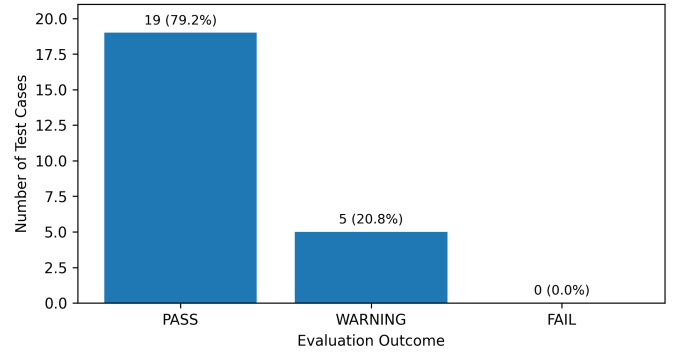


Fig. 3. Overall Evaluation Results

The language-wise results are presented in Table III. English, Tamil, Hindi, Bengali, and Marathi produced PASS results for all three test cases. Kannada produced two PASS and one WARNING, while Telugu and Malayalam produced one PASS and two WARNING results each. These warning cases indicate that response completeness can vary for some regional-language outputs under the current local LLM configuration. They should be interpreted within the scope of the evaluated test set and not as a general ranking of language performance.

TABLE III
LANGUAGE-WISE EVALUATION RESULTS

Language	PASS	WARNING	FAIL	Total
English	3	0	0	3
Tamil	3	0	0	3
Hindi	3	0	0	3
Telugu	1	2	0	3
Kannada	2	1	0	3
Malayalam	1	2	0	3
Bengali	3	0	0	3
Marathi	3	0	0	3
Total	19	5	0	24

The retrieval diagnostics showed that semantic retrieval generally required substantially less time than local LLM

generation. Most observed retrieval operations were completed within approximately 0.24–0.63 s, with one measured retrieval time of 8.10 s. The retrieved chunks were retained for inspection, while context selection reduced unnecessary or weakly relevant content before generation. A formal Recall@K or Mean Reciprocal Rank value is not reported because the current evaluation set does not contain explicit relevance labels required for these metrics.

The latency measurements further showed that local LLM inference was the dominant contributor to end-to-end response time. The observed first-generation inference times in the available diagnostic measurements ranged from approximately 52.63 to 84.58 s, while the average total response time across the current 24-test evaluation was approximately 102 s. These measurements are specific to the local hardware and inference configuration and should not be considered universal performance values. The results suggest that the local LLM inference constitutes the larger portion of the end-to-end latency in the current implementation, and semantic retrieval typically takes much less time.

D. Reliability and Discussion

The system can handle unsupported queries, incomplete/corrupted documents, generation failures, invalid metadata, and incomplete responses. When the information cannot be found in the selected documents, the system does not fall back to unsupported model knowledge, but reports the information is not available. The evaluation results demonstrate language fidelity, grounding, and source traceability across the supported eight languages, although additional work is needed to enhance response completeness for Telugu, Kannada, and Malayalam.

The main bottleneck in performance is LLM inference within the local inference engine. This evaluation used only 24 test cases and performance is hardware-dependent, so these results reflect the system’s performance in the tested environment, but not generally for the full set of regional-language queries and document types.

VI. CONCLUSION AND FUTURE SCOPE

DesiQuery is a locally deployed multilingual RAG framework for Document Grounded Question Answering across 8 Indian languages, built using BAAI/bge-m3, ChromaDB, Ollama model of Llama 3, along with source, page, chunk and similarity traceability. The evaluation of 24 test cases resulted in 19 PASS, 5 WARNING and 0 FAIL cases, with successful language, grounding and source-traceability validation for the test set. The results validate the proposed workflow and identify the local LLM inference latency and limitations in response completion as areas for future enhancements.

Future work will focus on expanding the multilingual evaluation dataset, creating formally annotated retrieval benchmarks, and conducting ablation studies for retrieval depth and context filtering. Further improvements will investigate optimized or quantized local models to reduce inference latency, improved processing of scanned PDFs, tables and complex

document structures, and language-specific evaluation and optimization for regional-language responses. These extensions can improve the retrieval reliability, multilingual robustness, and computational efficiency of DesiQuery.

REFERENCES

- [1] T. F. Khan, M. Arslan, F. Alturise, A. H. Almutairi, and M. T. Raza, “Uqa-rag: Enhancing urdu question-answer retrieval with semantic similarity and llms,” 2026.
- [2] S. Amal, A. R. Niranjana, and M. G. Thushara, “EduRag: A multimodal explainable rag system with deterministic token grounding,” 2026.
- [3] M. Hindi, L. Mohammed, O. Maaz, and A. Alwarafy, “Enhancing the precision and interpretability of retrieval-augmented generation (rag) in legal technology: A survey,” *IEEE Access*, 2025.
- [4] J. Xu, H. Zhang, H. Zhang, J. Lu, and G. Xiao, “Chattf: A knowledge graph-enhanced intelligent q&a system for mitigating factuality hallucinations in traditional folklore,” *IEEE Access*, vol. 12, 2024.
- [5] B. Saha, U. Saha, and M. Z. Malik, “Quim-rag: Advancing retrieval-augmented generation with inverted question matching for enhanced qa performance,” *IEEE Access*, vol. 12, 2024.
- [6] A. Onan, A. H. Nasution, T. Celikten, and P. Cetin, “Comparing chatgpt, gemini, and emerging llms in low-resource educational settings: Reasoning quality, consistency, and explainability,” *IEEE Access*, 2026.
- [7] A. Perevalov, A. Gashkov, M. Eltsova, and A. Both, “Sparql query candidate filtering for improving the quality of multilingual question answering over knowledge graphs using language models,” *Journal of Web Engineering*, vol. 24, 2025.
- [8] A. V. A. Sinthusha, E. Y. A. Charles, and R. Weerasinghe, “Machine reading comprehension for the tamil language with translated squad,” 2025.
- [9] M. Ali, R. Speck, H. M. Zahera, M. Saleem, D. Moussallem, and A.-C. N. Ngomo, “Multilingual relation extraction: A survey,” *IEEE Access*, 2025.
- [10] K. A. Hambarde and H. Proença, “Information retrieval: Recent advances and beyond,” 2023.
- [11] Z. Hu, P. Yang, F. Liu, Y. Meng, and X. Liu, “Prompting large language models with knowledge-injection for knowledge-based visual question answering,” *Big Data Mining and Analytics*, vol. 7, no. 3, 2024.
- [12] L. Kong, X. Zhong, J. Chen, H. Fu, and Y. Wang, “Multi-perspective consistency checking for large language model hallucination detection: A black-box zero-resource approach,” *Frontiers of Information Technology & Electronic Engineering*, 2025.
- [13] S. Behmanesh, A. Talebpour, M. Shamsfard, and M. M. Jafari, “A novel open-domain question answering system on curated and extracted knowledge bases with consideration of confidence scores in existing triples,” 2024.
- [14] M. L. L. de Oliveira and J. de Jesus Fiais Cerqueira, “Resource-constrained evaluation of quantized local llms for retrieval-augmented generation on a reduced-corpus setting,” *IEEE Access*, 2026.
- [15] J. Choi, S. Hong, S. Hong, J. Park, and E.-S. Jung, “Toward generating quality test questions and answers using quantized low-rank adapters in llms,” 2025.
- [16] P.-J. Liao, H.-Y. Lee, and H.-M. Wang, “Cross-attention reprogramming for asr: Bridging discrete speech units and pretrained language models,” 2026.